

Sage Serve

MLOPS - Project

Serving LLMs at Scale

Microsoft 365 serves millions of LLM requests across multiple data centers.

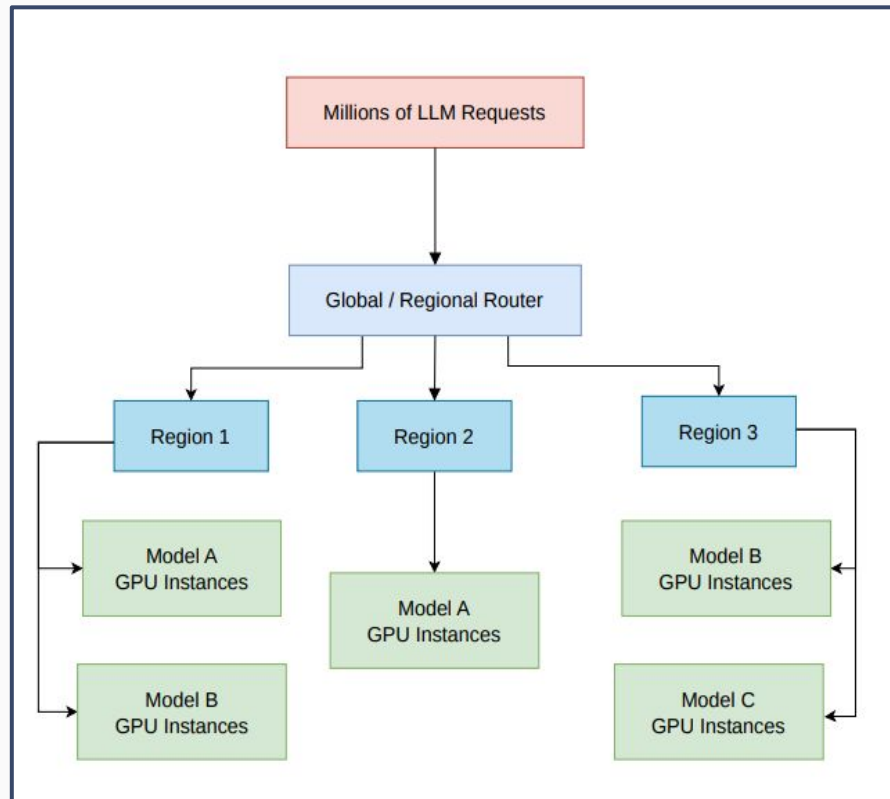
At any point, the system must decide:

- **Which model** should run?
- **Where** should it run?
- **How many GPU instances** should be active?

Note that demand changes over the following,

Time, Region, Model, Workload

GPUs are expensive resources.



Not All Requests Are Equal

Different workloads have very different latency requirements.

Workload	Requirement	Behavior
IW-F	< 1 sec TTFT	Very fast interactive
IW-N	< 1 min TTFT	Normal interactive
NIW	Deadline in hours	Non-interactive

IW workloads are latency sensitive. NIW workloads are flexible.

The First Problem: Reactive Autoscaling

Traditional reactive autoscaling

1. Monitor GPU utilization or incoming load
2. Detect a demand spike
3. Provision a new GPU instance
4. Load the LLM onto the GPU
5. Instance becomes ready

Under-provisioning

GPU isn't ready when demand arrives ,
leads to **SLA violation**

Over-provisioning

GPUs are started in anticipation of demand
, **Idle GPU-hours / wasted capacity**

The problem is that **LLM instances have long cold-start times**, because provisioning the VM and loading large model weights can take **minutes to hours**.

**Reactive scaling responds to demand;
LLM serving needs to anticipate it.**

Demand Is Predictable

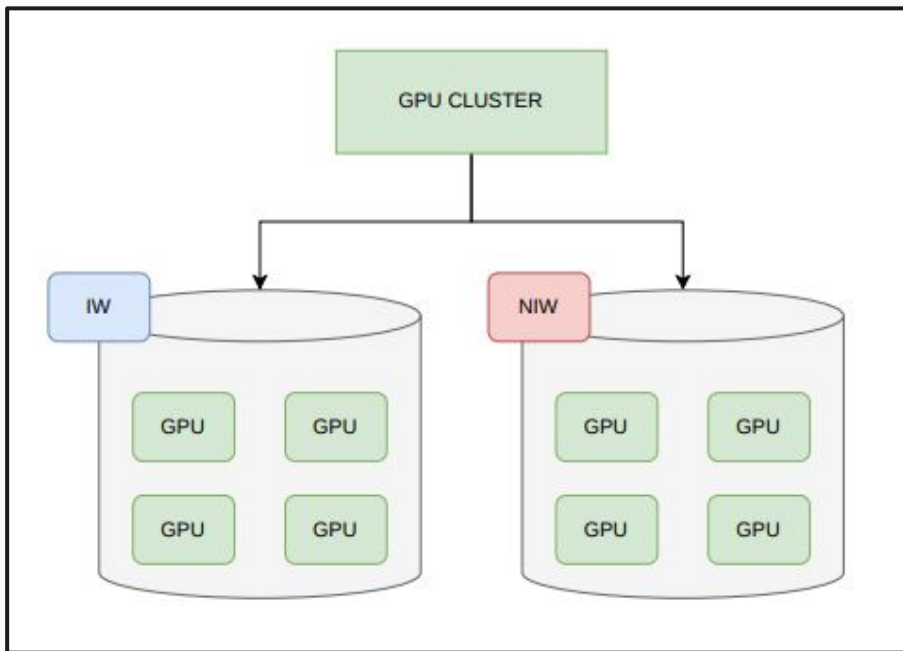
The authors analyze production LLM workloads and find that **demand is not completely random**.

Workload	Requests	Observed periodicity
IW-F	strong temporal patterns	0.7–0.95
IW-N	moderate periodicity	0.3–0.8
NIW	much less predictable	0–0.286

NIW has relaxed deadlines, so it can use spare capacity when available.

The Second Problem: Siloed GPU Pools

Interactive and non-interactive workloads are traditionally served by separate GPU pools.



The problem

Demand is not synchronized across the two pools.

Uneven Demand Example

Interactive demand increases, heavily loading the IW pool, while NIW demand remains low.

One pool can be **overloaded** while another has **unused** GPU capacity

Unified GPU Pool

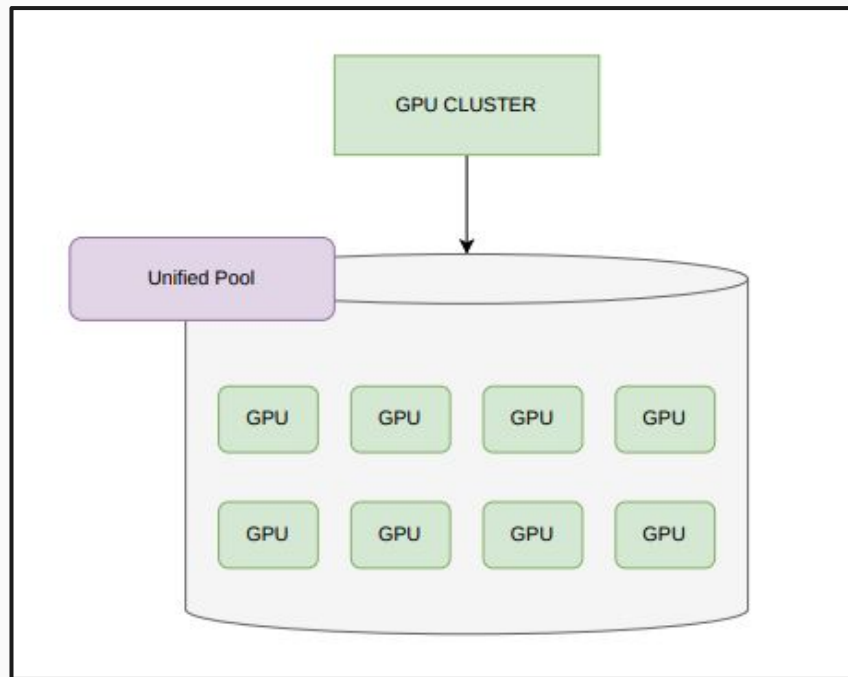
Share GPU capacity while protecting latency-sensitive workloads.

Instead of maintaining independent capacity for each workload, the same GPU pool can be dynamically shared.

IW gets priority
NIW fills available capacity

Experimental Result

34.5% fewer instance-hours



The Capacity Planning Problem

Given the expected workload, how should we provision GPU capacity?

DEMAND

How much traffic
is expected?

CAPACITY

How many
instances are
needed?

SLA

Can requests
meet their
deadlines?

COST

How do we avoid
unnecessary
GPUs?

Find the GPU allocation that satisfies workload SLAs while minimizing resource usage.